

Manuale Deploy — 108 AI Platform su Hetzner CX23

Target: dev italiano che non ha mai deployato su VPS

Server: Hetzner CX23 — 4 vCPU, 8 GB RAM, 80 GB SSD, Ubuntu 24.04

Costo stimato: ~7 EUR/mese

Tempo stimato setup completo: 2-3 ore

Indice

- Acquisto server Hetzner
- Setup iniziale del server
- Dominio e DNS
- Clone repo e configurazione .env
- Docker Compose in produzione
- SSL con Traefik e Let's Encrypt
- Deploy applicazioni (Gateway, Dashboard, Client)
- Desktop Agent — Build e distribuzione
- Git Autodeploy con webhook
- Backup automatico
- Monitoring
- RAM Budget
- Troubleshooting
- Security Hardening

1. Acquisto server Hetzner

Piano consigliato: CX23

Vai su console.hetzner.com e crea un account.

Perché CX23 e non CX22?

Il CX22 ha 4 GB RAM — troppo poco per Far girare PostgreSQL, Redis, Qdrant, Neo4j, LiteLLM e Traefik insieme. Il CX23 con 8 GB ha un margine di sicurezza adeguato (vedi RAM Budget per il breakdown esatto).

Configurazione da scegliere:

Campo	Valore
Location	Falkenstein (fsn1) — Europa, bassa latenza per clienti IT
Image	Ubuntu 24.04 LTS
Type	CX23 (4 vCPU, 8 GB RAM, 80 GB SSD)
Networking	IPv4 + IPv6 (inclusi nel prezzo)
SSH Key	Aggiungila qui (vedi sotto come crearla)
Backups	Opzionale — +20% (~1.40 EUR/mese) — consigliato
Name	aia-prod-01

Costo: ~6.90 EUR/mese IVA inclusa.

Creare la SSH key (se non ce l'hai)

Sul tuo Mac/Linux/WSL:

```
# Genera una nuova chiave SSH (salta se ne hai già una)
ssh-keygen -t ed25519 -C "aia-hetzner" -f ~/.ssh/aia_hetzner

# Mostra la chiave pubblica da incollare su Hetzner
cat ~/.ssh/aia_hetzner.pub
```

Su Windows (PowerShell):

```
ssh-keygen -t ed25519 -C "aia-hetzner" -f "$env:USERPROFILE\.ssh\ai_hetzner"
Get-Content "$env:USERPROFILE\.ssh\ai_hetzner.pub"
```

Copia l'output e incollalo nel campo "SSH Keys" durante la creazione del server su Hetzner.

Primo accesso

Dopo ~30 secondi dalla creazione il server è pronto. Prendi l'IP dalla console Hetzner.

```
# Primo accesso come root
ssh -i ~/.ssh/aia_hetzner root@<IP_SERVER>

# Accetta l'host key quando chiede "Are you sure?"
```

2. Setup iniziale del server

Tutti i comandi seguenti si eseguono sul server, dopo essersi connessi via SSH come root.

2.1 Aggiornamento sistema

```
apt update && apt upgrade -y
apt install -y curl git wget htop unzip ufw fail2ban
```

2.2 Creare utente non-root

Non lavorare mai come root in produzione. Crea un utente dedicato:

```
# Crea l'utente 'deploy'
adduser deploy
# Ti chiede una password — metterne una forte e salvala in un password manager

# Aggiungi ai sudoers
usermod -aG sudo deploy

# Copia la tua SSH key sull'utente deploy
mkdir -p /home/deploy/.ssh
cp /root/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
```

Ora apri un nuovo terminale e verifica che il login funzioni:

```
ssh -i ~/.ssh/aia_hetzner deploy@<IP_SERVER>
```

Se funziona, torna al terminale root e disabilita il login root via SSH:

```
# Sempre dal terminale root
sed -i 's/##PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config
systemctl restart sshd
```

2.3 Configurare UFW (firewall)

UFW è il firewall di Ubuntu. La regola di base: blocca tutto, apri solo quello che serve.

```
# Abilita UFW con policy di default
ufw default deny incoming
ufw default allow outgoing

# Apri le porte necessarie
ufw allow 22/tcp # SSH — FONDAMENTALE, non dimenticarla!
ufw allow 80/tcp # HTTP (Traefik lo redirige a HTTPS)
ufw allow 443/tcp # HTTPS

# Abilita il firewall
ufw enable
# Risponde "Command may disrupt existing ssh connections. Proceed with operation (y|n)?" → y

# Verifica lo stato
ufw status verbose
```

Attenzione: le porte interne (PostgreSQL 5432, Redis 6379, Qdrant 6333, Neo4j 7474/7687, LiteLLM 4000) NON vanno aperte al pubblico. Queste porte sono accessibili solo all'interno della rete Docker. Tienile chiuse.

2.4 Swap file (2 GB)

Con 8 GB di RAM e molti container, aggiungere 2 GB di swap ti salva nei momenti di picco:

```
# Crea un file di swap da 2 GB
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile

# Rendi lo swap permanente dopo i reboot
echo '/swapfile none swap sw 0 0' >> /etc/fstab

# Ottimizza per non usare swap troppo presto (0=mai, 10=quasi mai, 60=default)
echo 'vm.swappiness=10' >> /etc/sysctl.conf
sysctl -p

# Verifica
free -h
```

Output atteso:

```
      total    used    free
Mem:    7.8Gi    ...    ...
Swap:   2.0Gi     0B    2.0Gi
```

2.5 Installare Docker

```
# Installa Docker CE via script ufficiale
curl -fsSL https://get.docker.com | sh

# Aggiungi l'utente 'deploy' al gruppo docker (evita sudo per ogni comando docker)
usermod -aG docker deploy

# Abilita Docker all'avvio
systemctl enable docker
systemctl start docker

# Verifica
docker --version
docker compose version
```

Importante: dopo `usermod -aG docker deploy`, l'utente deve fare logout e login perché il gruppo sia attivo:

```
# Disconnettiti e riconnettiti come 'deploy'
exit
ssh -i ~/.ssh/aia_hetzner deploy@<IP_SERVER>

# Verifica che docker funzioni senza sudo
docker ps
```

2.6 Configurare il timezone

```
timedatectl set-timezone Europe/Rome
timedatectl status
```

3. Dominio e DNS

Hai bisogno di un dominio. Esempi:

- `api.108ai.dev` — per l'API principale
- `llm.108ai.dev` — per LiteLLM Gateway
- `traefik.108ai.dev` — per il dashboard Traefik

3.1 Record DNS da creare

Nel pannello del tuo registrar (Cloudflare, Namecheap, Aruba, ecc.) crea questi record A:

Nome	Tipo	Valore	TTL
<code>api</code>	<code>A</code>	<code><IP_SERVER></code>	<code>300</code>
<code>llm</code>	<code>A</code>	<code><IP_SERVER></code>	<code>300</code>
<code>traefik</code>	<code>A</code>	<code><IP_SERVER></code>	<code>300</code>
<code>qdrant</code>	<code>A</code>	<code><IP_SERVER></code>	<code>300</code>

Se usi Cloudflare, disabilita il proxy (nuvola arancione → grigia) per questi record. Let's Encrypt non funziona con il proxy Cloudflare attivo durante la prima emissione del certificato.

3.2 Verifica propagazione DNS

```
# Dal tuo PC locale — attendi che il DNS si propaghi (di solito 5-30 minuti)
dig api.tuodominio.com +short
# Deve rispondere con l'IP del server
```

Puoi anche usare dnschecker.org per verificare la propagazione globale.

4. Clone repo e configurazione .env

Da questo punto lavori sempre come utente `deploy` sul server.

4.1 Scegliere la directory di deploy

```
# Crea la directory per il progetto
sudo mkdir -p /opt/aia-platform
sudo chown deploy:deploy /opt/aia-platform
cd /opt/aia-platform
```

4.2 Clone del repository

```
# Se il repo è privato su GitHub, prima configura una Deploy Key
# (vedi sezione 7 — Git Autodeploy per i dettagli completi)

# Clone pubblico
git clone https://github.com/tuo-org/aia-platform.git .

# Oppure con SSH (repo privato)
git clone git@github.com:tuo-org/aia-platform.git .
```

4.3 Configurare il file .env

```
# Copia il template
cp .env.example .env

# Modifica con il tuo editor preferito
nano .env
```

Spiegazione di ogni variabile:

Database PostgreSQL

```
# Utente del database
POSTGRES_USER=aia

# Password del database — usa una password forte, minimo 32 caratteri
# Genera una password sicura con: openssl rand -base64 32
POSTGRES_PASSWORD=<GENERA_CON_OPENSSL>

# Nome del database
POSTGRES_DB=aia_platform

# URL di connessione completo — aggiorna con la password che hai scelto sopra
DATABASE_URL=postgresql://aia:<POSTGRES_PASSWORD>@postgres:5432/aia_platform
```

Redis

```
# URL interno Docker — non cambiare a meno che non rinomini il container
REDIS_URL=redis://redis:6379
```

Qdrant (Vector DB)

```
# URL interno Docker — non cambiare
QDRANT_URL=http://qdrant:6333
```

LiteLLM Gateway

```
# Master key per l'AI Gateway — genera con: openssl rand -base64 32
# Deve iniziare con "sk-" per compatibilità OpenAI
LITELLM_MASTER_KEY=sk-108ai-<GENERA_CON_OPENSSL>

# URL interno Docker — non cambiare
LITELLM_URL=http://litellm:4000
```

Chiavi API Provider AI

```
# DeepSeek — provider principale (economico, alta qualità)
# Ottieni su: https://platform.deepseek.com/api-keys
DEEPSEEK_API_KEY=sk-xxx

# Alibaba DashScope — fallback e embedding (free tier 1M token/mese)
# Ottieni su: https://dashscope-intl.aliyuncs.com/ → API Keys
DASHSCOPE_API_KEY=sk-xxx

# OpenAI — solo per embedding di fallback (opzionale)
OPENAI_API_KEY=sk-xxx

# Anthropic — tier premium opzionale (Claude)
ANTHROPIC_API_KEY=sk-ant-xxx
```

Nota costi: Con DeepSeek come provider primario, il costo per uso moderato è ~\$3-10/mese. Vedi i commenti nel file `infrastructure/litellm/config.yaml` per il pricing aggiornato.

Traefik e SSL

```
# Il tuo dominio principale (senza sottodomain, senza https://)
DOMAIN=108ai.dev

# Email per Let's Encrypt — riceve notifiche di scadenza
ACME_EMAIL=admin@108vision.it

# Credenziali dashboard Traefik — formato htpasswd
# ATTENZIONE: i $ vanno raddoppiati nel file .env
# Genera con: htpasswd -nb admin tuapassword
# Poi sostituisci ogni $ con $$
TRAEFIK_DASHBOARD_AUTH=admin:$$apr1$$HASH_GENERATO
```

Per generare `TRAEFIK_DASHBOARD_AUTH`:

```
# Installa apache2-utils se non ce l'hai
sudo apt install -y apache2-utils

# Genera l'hash
htpasswd -nb admin tuapassword_sicura
# Output esempio: admin:$apr1$xyz$hash123

# Nel .env, ogni singolo $ diventa $$
# admin:$apr1$xyz$hash123 → admin:$$apr1$$xyz$$hash123
```

Applicazione

```
# Ambiente (non cambiare in prod)
NODE_ENV=production

# JWT Secret — genera con: openssl rand -base64 48
JWT_SECRET=<GENERA_CON_OPENSSL_48CHARS>

# Porta dell'API gateway (non esposta direttamente — passa per Traefik)
PORT=3000
```

Web Search (opzionale)

```
# Scegli uno dei due se vuoi il tool web.search
BRAVE_SEARCH_API_KEY= # https://brave.com/search/api/ — free tier 2000 req/mese
TAVILY_API_KEY= # https://tavily.com/ — free tier disponibile
```

4.4 Proteggere il file .env

```
# Solo l'utente deploy può leggere il .env
chmod 600 .env

# Verifica
ls -la .env

# Deve mostrare: -rw----- 1 deploy deploy
```

4.5 Generare le password sicure — comandi rapidi

```
# Password PostgreSQL (32 caratteri)
openssl rand -base64 32

# LiteLLM Master Key
echo "sk-108ai-$(openssl rand -base64 24)"

# JWT Secret (48 caratteri)
openssl rand -base64 48
```

5. Docker Compose in produzione

5.1 Creare la directory per il certificato Traefik

Il volume traefik-certs viene gestito da Docker, ma il file acme.json deve avere i permessi giusti:

```
# Crea il volume esplicitamente
docker volume create aia-platform_traefik-certs

# Trova il path del volume
docker volume inspect aia-platform_traefik-certs --format '{{ .Mountpoint }}'
# Es: /var/lib/docker/volumes/aia-platform_traefik-certs/_data

# Crea il file acme.json con i permessi corretti
sudo touch /var/lib/docker/volumes/aia-platform_traefik-certs/_data/acme.json
sudo chmod 600 /var/lib/docker/volumes/aia-platform_traefik-certs/_data/acme.json
```

5.2 Verificare la configurazione prima di avviare

```
cd /opt/aia-platform

# Verifica che il docker-compose.yml sia valido
docker compose config

# Output atteso: stampa tutta la configurazione espansa senza errori
```

5.3 Avviare i servizi in ordine corretto

Avvia prima i servizi di infrastruttura (database), poi gli applicativi:


```
cd /opt/aia-platform

# Step 1: Avvia database e cache
docker compose up -d postgres redis

# Aspetta che siano healthy
docker compose ps
# Attendi che postgres e redis mostrino "healthy"

# Step 2: Avvia Qdrant e Neo4j
docker compose up -d qdrant neo4j

# Step 3: Avvia tutto il resto (LiteLLM + Traefik)
docker compose up -d

# Verifica che tutti i container siano up
docker compose ps
```

5.4 Verificare lo stato dei container

```
# Stato sintetico
docker compose ps

# Output atteso:
# NAME          IMAGE                STATUS    PORTS
# aia-litellm    ghcr.io/berriai/litellm Up (healthy) 0.0.0.0:4000->4000/tcp
# aia-neo4j      neo4j:5-community    Up (healthy) 0.0.0.0:7474->7474/tcp
# aia-postgres   pgvector/pgvector:pg16 Up (healthy) 0.0.0.0:5432->5432/tcp
# aia-qdrant     qdrant/qdrant:latest Up (healthy) 0.0.0.0:6333->6333/tcp
# aia-redis      redis:7-alpine        Up (healthy) 0.0.0.0:6379->6379/tcp
# aia-traefik    traefik:v3.1          Up         0.0.0.0:80->80/tcp
```

5.5 Vedere i log

```
# Log di tutti i container in real-time
docker compose logs -f

# Log di un singolo container
docker compose logs -f litellm
docker compose logs -f postgres
docker compose logs -f traefik

# Ultimi 100 log di tutti
docker compose logs --tail=100
```

6. SSL con Traefik e Let's Encrypt

Traefik gestisce SSL in modo completamente automatico. Non devi fare nulla di speciale — ma devi capire come funziona per fare troubleshooting.

Come funziona

- Traefik ascolta sulla porta 80 (HTTP)
- Quando arriva una richiesta per `api.108ai.dev`, Traefik fa l'HTTP Challenge con Let's Encrypt
- Let's Encrypt verifica che `api.108ai.dev` punti a questo server (per questo il DNS deve essere pronto prima)

- Let's Encrypt rilascia il certificato TLS, che Traefik salva in /letsencrypt/acme.json
- Traefik redirige automaticamente tutto il traffico HTTP → HTTPS

Verificare che il certificato sia stato emesso

```
# Aspetta 30-60 secondi dall'avvio di Traefik, poi:
curl -v https://api.108ai.dev/health 2>&1 | grep -E "SSL|TLS|certificate|issuer"

# Oppure controlla i log di Traefik per vedere la negoziazione ACME
docker compose logs traefik | grep -i "acme|certificate|letsencrypt"
```

Limiti di Let's Encrypt da conoscere

- Rate limit: max 5 certificati per dominio per 7 giorni. Se fai molti restart durante il setup, rischi di esaurirli.
- Soluzione: se stai testando, commenta temporaneamente il certresolver nel compose e usa HTTP. Aggiungi il resolver solo quando il dominio è stabile.

Dashboard Traefik

Accessibile su <https://traefik.108ai.dev> con le credenziali configurate in `TRAEFIK_DASHBOARD_AUTH`.

Mostra:

- Tutti i router attivi
- Health dei servizi
- Metriche Prometheus integrate

7. Deploy applicazioni (Gateway, Dashboard, Client)

Il docker-compose base contiene solo l'infrastruttura (DB, cache, AI, proxy). I servizi applicativi vanno aggiunti come container separati, che vengono buildati direttamente dal codice sorgente.

7.1 docker-compose.apps.yml — Container applicativi

Crea un file override per i container applicativi. Si usa un file separato per evitare di mischiare infrastruttura (immagini prebuilt) con applicazioni (build dal sorgente).

```

cat > /opt/aia-platform/docker-compose.apps.yml << 'EOF'
version: "3.9"

services:
  gateway:
    build:
      context: .
      dockerfile: apps/gateway/Dockerfile
    container_name: aia-gateway
    restart: unless-stopped
    environment:
      NODE_ENV: production
      PORT: 3000
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      REDIS_URL: redis://redis:6379
      QDRANT_URL: http://qdrant:6333
      LITELLM_URL: http://litellm:4000
      LITELLM_MASTER_KEY: ${LITELLM_MASTER_KEY}
      NEO4J_URI: bolt://neo4j:7687
      NEO4J_USER: neo4j
      NEO4J_PASSWORD: ${NEO4J_PASSWORD:-neo4j_dev_password}
      JWT_SECRET: ${JWT_SECRET}
      CORS_ALLOWED_ORIGINS: https://${DOMAIN},https://app.${DOMAIN}
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - aia-network
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.gateway.rule=Host(`${api}.${DOMAIN}`)"
      - "traefik.http.routers.gateway.entrypoints=websecure"
      - "traefik.http.routers.gateway.tls.certresolver=letsencrypt"
      - "traefik.http.services.gateway.loadbalancer.server.port=3000"
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://localhost:3000/health/live').then(r=>r.ok?process.exit(0);process.exit(
      interval: 30s
      timeout: 5s
      start_period: 15s
      retries: 3
    deploy:
      resources:
        limits:
          memory: 512M

  dashboard:
    build:
      context: apps/dashboard
      dockerfile: Dockerfile
    container_name: aia-dashboard
    restart: unless-stopped
    depends_on:
      - gateway

```

7.2 Aggiungere i DNS per le applicazioni

Nel pannello del registrar, aggiungi questi record A (come fatto alla sezione 3):

Nome	Tipo	Valore	Ruolo
------	------	--------	-------

app	A	<IP_SERVER>	Dashboard admin
chat	A	<IP_SERVER>	Widget chat client

7.3 Aggiungere le variabili .env mancanti

```
# Aggiungi al file .env queste variabili se non presenti:
cat >> /opt/aia-platform/.env << 'EOF'

# --- App Configuration ---
# Password Neo4j (deve corrispondere a quella nel docker-compose.yml)
NEO4J_PASSWORD=<GENERA_CON_OPENSSL>

# Origini CORS permesse (separate da virgola)
# Usato dal gateway per validare le richieste cross-origin
# CORS_ALLOWED_ORIGINS viene costruito automaticamente dal DOMAIN nel compose
EOF
```

7.4 Avviare le applicazioni

```
cd /opt/aia-platform

# Build e avvio (la prima volta ci vogliono 2-5 minuti per il build)
docker compose -f docker-compose.yml -f docker-compose.apps.yml up -d --build

# Verifica lo stato
docker compose -f docker-compose.yml -f docker-compose.apps.yml ps
```

Output atteso:

NAME	IMAGE	STATUS	PORTS
aia-gateway	aia-platform-gateway	Up (healthy)	3000/tcp
aia-dashboard	aia-platform-dashboard	Up	80/tcp
aia-client	aia-platform-client	Up	80/tcp
aia-litellm	ghcr.io/berriai/litellm	Up (healthy)	4000/tcp
...			

7.5 Verificare che tutto funzioni

```
# 1. Health check del gateway
curl -s https://api.108ai.dev/health/live
# Atteso: {"status":"ok"}

# 2. Dashboard raggiungibile
curl -s -o /dev/null -w "%{http_code}" https://app.108ai.dev
# Atteso: 200

# 3. Client raggiungibile
curl -s -o /dev/null -w "%{http_code}" https://chat.108ai.dev
# Atteso: 200

# 4. API auth funziona
curl -s https://api.108ai.dev/api/auth/health
# Atteso: {"status":"ok"} o simile
```

7.6 Come funziona il routing in produzione

```
Browser → https://app.108ai.dev
→ Traefik (SSL) → aia-dashboard (nginx:80)
→ Le chiamate /api/* vengono proxate da nginx → aia-gateway:3000
```

```
Browser → https://api.108ai.dev
→ Traefik (SSL) → aia-gateway:3000 (diretto)
```

Il nginx inside il container dashboard ha un proxy_pass `http://gateway:3000/api/` che risolve gateway via la Docker network interna. In produzione, le SPA (dashboard e client) servono file statici e proxano le API verso il container gateway.

7.7 Aggiornare deploy.sh per includere le app

Modifica lo script `/opt/aia-platform/deploy.sh` per usare entrambi i compose file:

```
# Nella sezione "Rebuilding containers..." del deploy.sh, sostituisci:
# docker compose up -d --build
# Con:
docker compose -f docker-compose.yml -f docker-compose.apps.yml pull 2>&1 | tee -a "$LOG_FILE"
docker compose -f docker-compose.yml -f docker-compose.apps.yml up -d --build 2>&1 | tee -a "$LOG_FILE"
```

7.8 Variabili d'ambiente di produzione — checklist

Variable | Valore in produzione | Note

NODE_ENV	production	Disabilita dev tools, abilita cache
JWT_SECRET	48+ char random	Mai usare il default dev
CORS_ALLOWED_ORIGINS	https://108ai.dev,https://app.108ai.dev	Solo domini reali
NEO4J_PASSWORD	32+ char random	Password dedicata
LITELLM_MASTER_KEY	sk-108ai-<random>	Deve iniziare con sk-

8. Desktop Agent — Build e distribuzione

Il Desktop Agent (`108ai.exe`) e' un binario compilato con Bun che i clienti scaricano e installano. Il server Hetzner serve come host per il download.

8.1 Build del binario (sulla macchina di sviluppo)

```
cd apps/local-agent

# Installa Bun se non ce l'hai (https://bun.sh)
# Windows: powershell -c "irm bun.sh/install.ps1 | iex"
# Mac/Linux: curl -fsSL https://bun.sh/install | bash

# Build per tutte le piattaforme
bun build src/index.ts --compile --outfile dist/108ai          # Linux
bun build src/index.ts --compile --target=bun-windows-x64 --outfile dist/108ai.exe # Windows
bun build src/index.ts --compile --target=bun-darwin-x64 --outfile dist/108ai-macos # Mac Intel
bun build src/index.ts --compile --target=bun-darwin-arm64 --outfile dist/108ai-macos-arm # Mac ARM
```

8.2 Firmare il binario (Code Signing)

Senza firma, Windows Defender/SmartScreen bloccano l'exe al primo avvio.

Opzioni di certificato:

Provider	Costo/anno	Note
DigiCert	~\$400	Standard enterprise, riconosciuto ovunque
Sectigo (Comodo)	~\$200	Buon rapporto qualita'/prezzo
SSL.com	~\$250	EV Code Signing (immediata reputazione)
SignPath.io	Free per OSS	Se il progetto e' open-source

Procedura firma Windows (signtool):

```
# Richiede il Windows SDK installato
# Il certificato .pfx deve essere nel keystore o su file
signtool sign /f "108vision-codesign.pfx" /p "<password>" /tr http://timestamp.digicert.com /td sha256 /fd sha256 dist/1

# Verifica la firma
signtool verify /pa dist/108ai.exe
```

Procedura firma Mac (codesign):

```
# Richiede un Apple Developer ID ($99/anno)
codesign --sign "Developer ID Application: 108 Vision S.r.l." --timestamp --options runtime dist/108ai-macos
codesign --sign "Developer ID Application: 108 Vision S.r.l." --timestamp --options runtime dist/108ai-macos-arm

# Notarization (necessario per macOS Gatekeeper)
xcrun notarytool submit dist/108ai-macos --apple-id "dev@108vision.it" --team-id XXXXXXXXXX --password @keychain:AC_PASS
```

8.3 Generare checksum e caricare sul server

```
# Genera SHA-256 per ogni binario
sha256sum dist/108ai dist/108ai.exe dist/108ai-macos dist/108ai-macos-arm > dist/checksums.txt

# Copia i binari sul server
scp -i ~/.ssh/aia_hetzner dist/108ai* deploy@<IP_SERVER>:/opt/aia-platform/public/downloads/
scp -i ~/.ssh/aia_hetzner dist/checksums.txt deploy@<IP_SERVER>:/opt/aia-platform/public/downloads/
```

8.4 Configurare il gateway per servire i download

Il gateway espone un endpoint `/api/desktop-agent/download/:platform` che serve il binario corretto. Per i file statici di grandi dimensioni, conviene servirli direttamente da Traefik/nginx.

Crea la directory dei download sul server:

```
sudo mkdir -p /opt/aia-platform/public/downloads
sudo chown deploy:deploy /opt/aia-platform/public/downloads
```

Aggiungi un servizio file-server leggero nel docker-compose.apps.yml:

```

downloads:
  image: nginx:1.27-alpine
  container_name: aia-downloads
  restart: unless-stopped
  volumes:
    - /opt/aia-platform/public/downloads:/usr/share/nginx/html:ro
  networks:
    - aia-network
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.downloads.rule=Host(`dl.${DOMAIN}`)"
    - "traefik.http.routers.downloads.entrypoints=websecure"
    - "traefik.http.routers.downloads.tls.certresolver=letsencrypt"
    - "traefik.http.services.downloads.loadbalancer.server.port=80"
  deploy:
    resources:
      limits:
        memory: 32M

```

Aggiungi il record DNS:

Nome	Tipo	Valore
dl	A	<IP_SERVER>

8.5 Endpoint version check per auto-update

Il gateway ha già una route per la versione. In produzione il Desktop Agent chiama:

```

GET https://api.108ai.dev/api/desktop-agent/version
→ { "latest": "1.0.0", "url": "https://dl.108ai.dev/108ai.exe", "sha256": "abc123..." }

```

Per aggiornare una release:

- Build + firma i binari (8.1 + 8.2)
- Carica su /opt/aia-platform/public/downloads/ (8.3)
- Aggiorna la versione nel gateway (variabile env o DB)

8.6 Workflow di rilascio Desktop Agent

```

Sviluppo locale
→ bun build --compile (3 piattaforme)
→ signtool / codesign (firma)
→ sha256sum (checksum)
→ scp sul server in /opt/aia-platform/public/downloads/
→ Aggiorna version.json o env var DESKTOP_AGENT_VERSION
→ Gli agent già installati ricevono la notifica al prossimo check (ogni 6h)

```

9. Git Autodeploy con webhook

Il deploy automatico funziona così: push su GitHub/GitLab → webhook → script sul server → git pull + docker compose up --build.

7.1 Creare una Deploy Key per il repo privato

Se il repo è privato, il server ha bisogno di una SSH key dedicata per fare git pull:

```
# Sul server, come utente deploy
ssh-keygen -t ed25519 -C "aia-deploy-key" -f ~/.ssh/aia_deploy_key -N ""

# Mostra la chiave pubblica
cat ~/.ssh/aia_deploy_key.pub
```

Su GitHub:

- Vai su Repository → Settings → Deploy Keys → Add deploy key
- Incolla il contenuto di ~/.ssh/aia_deploy_key.pub
- Spunta "Allow write access" solo se necessario (di solito no)

Su GitLab:

- Repository → Settings → Repository → Deploy Keys

Configura SSH per usare questa chiave con GitHub:

```
cat >> ~/.ssh/config << 'EOF'
Host github.com
  IdentityFile ~/.ssh/aia_deploy_key
  IdentitiesOnly yes
EOF
```

Ora rifai il clone con SSH:

```
cd /opt
rm -rf aia-platform
git clone git@github.com:tuo-org/aia-platform.git aia-platform
```

7.2 Creare lo script deploy.sh


```

cat > /opt/aia-platform/deploy.sh << 'EOF'
#!/bin/bash
set -e

DEPLOY_DIR="/opt/aia-platform"
LOG_FILE="/var/log/aia-deploy.log"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

log() {
    echo "[${TIMESTAMP}] $1" | tee -a "$LOG_FILE"
}

log "=== Deploy started ==="

cd "$DEPLOY_DIR"

# Pull latest code
log "Pulling latest code..."
git pull origin main 2>&1 | tee -a "$LOG_FILE"

# Rebuild and restart containers (only changed images are rebuilt)
log "Rebuilding containers..."
docker compose pull 2>&1 | tee -a "$LOG_FILE"
docker compose up -d --build 2>&1 | tee -a "$LOG_FILE"

# Health check — aspetta che i container siano healthy
log "Waiting for health checks..."
sleep 15

UNHEALTHY=$(docker compose ps --format json | python3 -c "
import sys, json
containers = [json.loads(line) for line in sys.stdin if line.strip()]
unhealthy = [c['Name'] for c in containers if c.get('Health', '') == 'unhealthy']
print('\n'.join(unhealthy))
" 2>/dev/null || echo "")

if [ -n "$UNHEALTHY" ]; then
    log "ERROR: Unhealthy containers detected:"
    log "$UNHEALTHY"
    log "Deploy FAILED — check logs with: docker compose logs"
    exit 1
fi

# Rimuovi immagini non usate per liberare spazio
log "Cleaning up old images..."
docker image prune -f 2>&1 | tee -a "$LOG_FILE"

log "=== Deploy completed successfully ==="
EOF

chmod +x /opt/aia-platform/deploy.sh

```

Crea il file di log con i permessi giusti:

```

sudo touch /var/log/aia-deploy.log
sudo chown deploy:deploy /var/log/aia-deploy.log

```

7.3 Installare il webhook server

Useremo webhook — un tool leggero in Go che ascolta HTTP e lancia script:

```
# Installa webhook
sudo apt install -y webhook

# Verifica
webhook -version
```

7.4 Configurare il webhook

```
mkdir -p /opt/webhooks

# Genera un segreto sicuro per validare le richieste da GitHub
WEBHOOK_SECRET=$(openssl rand -hex 32)
echo "Salva questo segreto: $WEBHOOK_SECRET"

cat > /opt/webhooks/hooks.json << EOF
[
  {
    "id": "aia-deploy",
    "execute-command": "/opt/aia-platform/deploy.sh",
    "command-working-directory": "/opt/aia-platform",
    "response-message": "Deploy triggered",
    "trigger-rule": {
      "and": [
        {
          "match": {
            "type": "payload-hmac-sha256",
            "secret": "$WEBHOOK_SECRET",
            "parameter": {
              "source": "header",
              "name": "X-Hub-Signature-256"
            }
          }
        },
        {
          "match": {
            "type": "value",
            "value": "refs/heads/main",
            "parameter": {
              "source": "payload",
              "name": "ref"
            }
          }
        }
      ]
    }
  }
]
EOF

echo "Webhook secret: $WEBHOOK_SECRET" > /home/deploy/webhook-secret.txt
chmod 600 /home/deploy/webhook-secret.txt
```

7.5 Creare il servizio systemd per il webhook

```

sudo cat > /etc/systemd/system/aia-webhook.service << 'EOF'
[Unit]
Description=AIA Platform Webhook Server
After=network.target

[Service]
Type=simple
User=deploy
Group=deploy
ExecStart=/usr/bin/webhook -hooks /opt/webhooks/hooks.json -port 9000 -verbose
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable aia-webhook
sudo systemctl start aia-webhook
sudo systemctl status aia-webhook

```

7.6 Esporre il webhook tramite Traefik (HTTPS)

Invece di aprire la porta 9000 nel firewall, esponi il webhook via Traefik su HTTPS. Aggiungi questi label al servizio traefik nel docker-compose.yml, oppure configura il webhook direttamente:

```

# Apri la porta solo per il webhook (alternativa semplice: porta diretta)
sudo ufw allow 9000/tcp comment "Webhook server"

```

Oppure (consigliato — via Traefik) — aggiungi un container webhook nel compose:

```

# Da aggiungere al docker-compose.yml
webhook:
  image: almir/webhook
  container_name: aia-webhook
  restart: unless-stopped
  volumes:
    - /opt/webhooks:/hooks:ro
    - /opt/aia-platform:/opt/aia-platform:rw
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - /var/log/aia-deploy.log:/var/log/aia-deploy.log:rw
  command: -hooks /hooks/hooks.json -verbose
  networks:
    - aia-network
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.webhook.rule=Host(`deploy.${DOMAIN}`)"
    - "traefik.http.routers.webhook.entrypoints=websecure"
    - "traefik.http.routers.webhook.tls.certresolver=letsencrypt"
    - "traefik.http.services.webhook.loadbalancer.server.port=9000"

```

7.7 Configurare il webhook su GitHub

- Vai su Repository → Settings → Webhooks → Add webhook
- Payload URL: <https://deploy.108ai.dev/hooks/aia-deploy> (oppure <http://<IP>:9000/hooks/aia-deploy>)
- Content type: application/json

- Secret: il valore di WEBHOOK_SECRET generato al punto 7.4
- Trigger: "Just the push event"
- Active: spuntato

7.8 Test del deploy automatico

```
# Test manuale dello script deploy
/opt/aia-platform/deploy.sh

# Monitora i log del deploy
tail -f /var/log/aia-deploy.log

# Dopo aver configurato il webhook su GitHub, fai un push di test:
# git commit --allow-empty -m "test: trigger deploy" && git push
```

10. Backup automatico

8.1 Script di backup PostgreSQL

```
mkdir -p /opt/backups/postgres

cat > /opt/backups/backup-postgres.sh << 'EOF'
#!/bin/bash
set -e

BACKUP_DIR="/opt/backups/postgres"
DATE=$(date '+%Y-%m-%d_%H-%M')
FILENAME="$BACKUP_DIR/aia_platform_${DATE}.sql.gz"
RETENTION_DAYS=${BACKUP_RETENTION_DAYS:-30}

# Carica le variabili di ambiente
source /opt/aia-platform/.env

echo "[${DATE}] Starting PostgreSQL backup..."

# Esegui il dump dentro il container postgres
docker exec aia-postgres pg_dump \
-U "$POSTGRES_USER" \
-d "$POSTGRES_DB" \
--clean \
--if-exists \
--no-owner \
--no-acl \
| gzip > "$FILENAME"

SIZE=$(du -sh "$FILENAME" | cut -f1)
echo "[${DATE}] Backup completed: $FILENAME ($SIZE)"

# Rimuovi backup più vecchi di RETENTION_DAYS giorni
find "$BACKUP_DIR" -name "*.sql.gz" -mtime "+$RETENTION_DAYS" -delete
echo "[${DATE}] Old backups cleaned (retention: $RETENTION_DAYS days)"
EOF

chmod +x /opt/backups/backup-postgres.sh
```

8.2 Script di backup volumi Docker

```

cat > /opt/backups/backup-volumes.sh << 'EOF'
#!/bin/bash
set -e

BACKUP_DIR="/opt/backups/volumes"
DATE=$(date '+%Y-%m-%d_%H-%M')
RETENTION_DAYS=7 # I volumi sono più grandi, tieni meno copie

mkdir -p "$BACKUP_DIR"

echo "[${DATE}] Starting volumes backup..."

for VOLUME in aia-platform_qdrant-data aia-platform_neo4j_data aia-platform_redis-data; do
    FILENAME="$BACKUP_DIR/${VOLUME}_${DATE}.tar.gz"
    docker run --rm \
        -v "$VOLUME":/data:ro \
        -v "$BACKUP_DIR":/backup \
        alpine tar czf "/backup/${VOLUME}_${DATE}.tar.gz" -C /data .
    SIZE=$(du -sh "$FILENAME" | cut -f1)
    echo "[${DATE}] Volume backup: $FILENAME ($SIZE)"
done

# Cleanup
find "$BACKUP_DIR" -name "*.tar.gz" -mtime "+$RETENTION_DAYS" -delete
echo "[${DATE}] Volumes backup completed"
EOF

chmod +x /opt/backups/backup-volumes.sh
chown -R deploy:deploy /opt/backups

```

8.3 Configurare cron job

```

# Apri il crontab dell'utente deploy
crontab -e

```

Aggiungi queste righe:

```

# Backup PostgreSQL ogni giorno alle 03:00
0 3 * * * /opt/backups/backup-postgres.sh >> /var/log/aia-backup.log 2>&1

# Backup volumi ogni domenica alle 04:00
0 4 * * 0 /opt/backups/backup-volumes.sh >> /var/log/aia-backup.log 2>&1

```

```

# Crea il file di log
sudo touch /var/log/aia-backup.log
sudo chown deploy:deploy /var/log/aia-backup.log

# Test manuale del backup
/opt/backups/backup-postgres.sh
ls -lh /opt/backups/postgres/

```

8.4 Backup remoto su S3 (opzionale ma consigliato)

Se hai configurato le variabili `BACKUP_S3_BUCKET`, `AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY` nel `.env`:

```
# Installa AWS CLI
sudo apt install -y awscli

# Testa la connessione
aws s3 ls s3://$BACKUP_S3_BUCKET

# Aggiungi al backup-postgres.sh, dopo la riga del dump:
# aws s3 cp "$FILENAME" "s3://$BACKUP_S3_BUCKET/postgres/"
```

8.5 Procedura di restore PostgreSQL

```
# Lista i backup disponibili
ls -lh /opt/backups/postgres/

# Restore (ATTENZIONE: sovrascrive il database corrente!)
BACKUP_FILE="/opt/backups/postgres/aia_platform_2026-06-13_03-00.sql.gz"

source /opt/aia-platform/.env

# Disconnetti le connessioni attive e ripristina
docker exec -i aia-postgres psql -U "$POSTGRES_USER" -d postgres -c \
"SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname='$POSTGRES_DB' AND pid <> pg_backend_pid();"

zcat "$BACKUP_FILE" | docker exec -i aia-postgres psql \
-U "$POSTGRES_USER" \
-d "$POSTGRES_DB"

echo "Restore completato"
```

11. Monitoring

9.1 Monitoring live con docker stats

```
# Uso RAM e CPU di tutti i container in tempo reale
docker stats

# Solo le colonne che interessano, aggiornate ogni secondo
watch -n 1 'docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}"'
```

9.2 Log aggregation

```
# Tutti i log, ultimi 200 righe, con timestamp
docker compose -f /opt/aia-platform/docker-compose.yml logs --tail=200 --timestamps

# Filtra per livello di errore
docker compose -f /opt/aia-platform/docker-compose.yml logs | grep -iE "error|warn|fatal"

# Segui i log di litellm in real-time (utile per debugging AI calls)
docker compose -f /opt/aia-platform/docker-compose.yml logs -f litellm
```

9.3 Health check rapido di tutti i servizi

```

cat > /opt/aia-platform/health-check.sh << 'EOF'
#!/bin/bash

echo "=== AIA Platform Health Check ==="
echo "Data: $(date)"
echo ""

# Container status
echo "--- Container Status ---"
docker compose -f /opt/aia-platform/docker-compose.yml ps --format "table {{.Name}}\t{{.Status}}"
echo ""

# Disk usage
echo "--- Disk Usage ---"
df -h / | tail -1 | awk '{print "Root: " $3 " used / " $2 " total (" $5 " full)}'
du -sh /opt/backups/ 2>/dev/null && echo "Backups: $(du -sh /opt/backups/ | cut -f1)"
echo ""

# RAM usage
echo "--- Memory Usage ---"
free -h | grep Mem | awk '{print "RAM: " $3 " used / " $2 " total"}'
free -h | grep Swap | awk '{print "Swap: " $3 " used / " $2 " total"}'
echo ""

# Docker volumes
echo "--- Docker Volumes Size ---"
docker system df
echo ""

echo "=== Fine Health Check ==="
EOF

chmod +x /opt/aia-platform/health-check.sh

# Esegui subito
/opt/aia-platform/health-check.sh

```

9.4 Alerting base via email (opzionale)

```

# Installa msmtplib per inviare email
sudo apt install -y msmtplib msmtplib-mta

# Configura per Gmail/SMTP
cat > /etc/msmtplibrc << 'EOF'
defaults
auth      on
tls       on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile    /var/log/msmtplib.log

account    gmail
host       smtp.gmail.com
port       587
from       tua-email@gmail.com
user       tua-email@gmail.com
password    tua-app-password-gmail

account default : gmail
EOF

chmod 600 /etc/msmtplibrc

```

Script di alerting per container unhealthy:

```
cat > /opt/aia-platform/alert-check.sh << 'EOF'
#!/bin/bash

ALERT_EMAIL="admin@108vision.it"
UNHEALTHY=$(docker compose -f /opt/aia-platform/docker-compose.yml ps --format json \
| python3 -c "
import sys, json
for line in sys.stdin:
    if line.strip():
        c = json.loads(line)
        if c.get("Health", "") == 'unhealthy':
            print(c['Name'])
" 2>/dev/null)

if [ -n "$UNHEALTHY" ]; then
    echo "Container unhealthy su $(hostname) alle $(date):\n$UNHEALTHY" \
    | mail -s "[ALERT] AIA Platform — Container Down" "$ALERT_EMAIL"
fi
EOF

chmod +x /opt/aia-platform/alert-check.sh

# Aggiungi al crontab — controlla ogni 5 minuti
# */5 * * * * /opt/aia-platform/alert-check.sh
```

9.5 Metriche Traefik con Prometheus (avanzato)

Traefik espone già le metriche Prometheus sulla porta websecure. Se vuoi un dashboard Grafana, puoi aggiungere al docker-compose.yml:

```
# Aggiunta opzionale — solo se vuoi metriche grafiche
prometheus:
  image: prom/prometheus:latest
  container_name: aia-prometheus
  restart: unless-stopped
  volumes:
    - ./infrastructure/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus-data:/prometheus
  networks:
    - aia-network

grafana:
  image: grafana/grafana:latest
  container_name: aia-grafana
  restart: unless-stopped
  networks:
    - aia-network
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.grafana.rule=Host(`metrics.${DOMAIN}`)"
    - "traefik.http.routers.grafana.entrypoints=websecure"
    - "traefik.http.routers.grafana.tls.certresolver=letsencrypt"
```

12. RAM Budget

Con 8 GB di RAM, ecco la distribuzione realistica a regime (valori misurati con docker stats):

Componente | RAM stimata | Note

OS Ubuntu 24.04 | ~600 MB | Kernel + systemd + SSH + cron

Docker daemon | ~100 MB | Overhead runtime

Traefik v3 | ~50 MB | Reverse proxy, molto leggero

PostgreSQL 16 + pgvector | ~300-500 MB | Dipende dal carico e shared_buffers

Redis 7 Alpine | ~30-50 MB | Cache in-memory — cresce con i dati

Qdrant | ~200-400 MB | Dipende dagli indici vettoriali caricati

LiteLLM | ~400-512 MB | Hard limit configurato nel compose

Neo4j Community | ~400-500 MB | Heap 256 MB + pagecache 128 MB configurati

App gateway | ~200-300 MB | Node.js API (hard limit 512M)

Dashboard (nginx) | ~10-20 MB | File statici, trascurabile

Client (nginx) | ~10-20 MB | File statici, trascurabile

Downloads (nginx) | ~10-20 MB | Binari desktop agent

Webhook server | ~20 MB | Trascurabile

Subtotale | ~2.4-3.2 GB | Carico normale

Buffer disponibile | ~4.8-5.6 GB | Per picchi, swap coverage, crescita

Neo4j è configurato nel compose con limiti espliciti:

```
NEO4J_dbms_memory_heap_max_size: 256m
NEO4J_dbms_memory_pagecache_size: 128m
```

LiteLLM ha un hard limit:

```
deploy:
  resources:
    limits:
      memory: 512M
```

Conclusione: con 8 GB il CX23 regge comodamente il carico base. Se Neo4j o Qdrant iniziano a crescere molto con i dati, il primo upgrade è al CX33 (16 GB RAM, ~€14/mese).

Quando fare upgrade

```
# Monitora il trend della RAM nel tempo
watch -n 5 'free -h && docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.MemPerc}}"' | sort -k3 -rh'
```

Upgrade al CX33 quando: RAM used > 6.5 GB in condizioni normali (non picco).

13. Troubleshooting

Container non parte: "port already in use"

```
# Trova chi usa la porta
sudo ss -tln | grep :5432
sudo lsof -i :5432

# Se è PostgreSQL di sistema (non il container)
sudo systemctl stop postgresql
sudo systemctl disable postgresql
```

LiteLLM non si connette al DB

LiteLLM dipende da postgres e redis healthy. Se parte prima che siano pronti:

```
# Controlla i log
docker compose logs litellm | head -50

# Riavvia solo litellm (i db rimangono up)
docker compose restart litellm

# Aspetta 30 secondi, poi verifica
docker compose ps litellm
```

Let's Encrypt non emette il certificato

Causa più comune: il DNS non è propagato quando Traefik fa l'HTTP Challenge.

```
# Verifica che il dominio risolva correttamente dal server
curl -v http://api.108ai.dev 2>&1 | head -20

# Controlla i log di Traefik per errori ACME
docker compose logs traefik | grep -iE "acme|error|certificate"

# Se hai raggiunto il rate limit di Let's Encrypt (5 cert / 7 giorni),
# usa il server di staging per i test:
# Nel traefik.yml, sotto certificatesResolvers.letsencrypt.acme, aggiungi:
# caServer: "https://acme-staging-v02.api.letsencrypt.org/directory"
```

Database corrupted o schema non aggiornato

```
# Connettiti al database
docker exec -it aia-postgres psql -U aia -d aia_platform

# Controlla le tabelle
\d

# Esci
\q
```

"No space left on device"

```
# Controlla lo spazio disco
df -h

# Trova i file più grandi
du -sh /var/lib/docker/volumes/* | sort -h | tail -20

# Libera spazio da immagini e container non usati
docker system prune -a --volumes
# ATTENZIONE: rimuove anche i volumi non associati a container running
# Usa con cautela — prima fai backup!

# Versione sicura (solo immagini e container, NON volumi)
docker system prune -a
```

Il deploy automatico non si attiva

```
# Verifica che il webhook server sia attivo
sudo systemctl status aia-webhook

# Verifica i log del webhook
journalctl -u aia-webhook -f

# Testa manualmente l'endpoint
curl -X POST http://localhost:9000/hooks/aia-deploy

# Verifica i log dei delivery webhook su GitHub
# Repository → Settings → Webhooks → clicca sul webhook → Recent Deliveries
```

Neo4j non parte: "heap too small"

```
# Controlla l'errore
docker compose logs neo4j | grep -iE "error|heap|memory"

# Se il server è sotto pressione RAM, riduci ulteriormente
# Nel docker-compose.yml:
# NEO4J_dbms_memory_heap_max__size: 128m
# NEO4J_dbms_memory_pagecache_size: 64m
```

Rollback manuale a una versione precedente

```
cd /opt/aia-platform

# Visualizza i commit recenti
git log --oneline -10

# Torna a un commit specifico (es. abc1234)
git checkout abc1234

# Riavvia con la versione precedente
docker compose up -d --build

# Per tornare al branch main
git checkout main
```

Reset completo (nucleare — solo in emergenza)

```
# ATTENZIONE: questo distrugge tutti i dati!
# Fai il backup prima.

cd /opt/aia-platform
docker compose down -v # il flag -v rimuove anche i volumi
docker system prune -a --volumes -f
docker compose up -d
```

14. Security Hardening

12.1 Fail2ban per bloccare brute force SSH

```
# fail2ban è già installato dal setup iniziale
```

```
# Configura la jail SSH
```

```
cat > /etc/fail2ban/jail.local << 'EOF'
```

```
[DEFAULT]
```

```
bantime = 3600
```

```
findtime = 600
```

```
maxretry = 5
```

```
[sshd]
```

```
enabled = true
```

```
port = ssh
```

```
logpath = %(sshd_log)s
```

```
backend = systemd
```

```
maxretry = 3
```

```
bantime = 86400
```

```
EOF
```

```
sudo systemctl restart fail2ban
```

```
sudo systemctl enable fail2ban
```

```
# Verifica lo stato
```

```
sudo fail2ban-client status
```

```
sudo fail2ban-client status sshd
```

12.2 Aggiornamenti automatici di sicurezza

```
sudo apt install -y unattended-upgrades apt-listchanges
```

```
# Configura per aggiornare solo i pacchetti di sicurezza
```

```
cat > /etc/apt/apt.conf.d/50unattended-upgrades << 'EOF'
```

```
Unattended-Upgrade::Allowed-Origins {
```

```
    "${distro_id}:${distro_codename}-security";
```

```
};
```

```
Unattended-Upgrade::AutoFixInterruptedDpkg "true";
```

```
Unattended-Upgrade::MinimalSteps "true";
```

```
Unattended-Upgrade::Remove-Unused-Dependencies "true";
```

```
Unattended-Upgrade::Automatic-Reboot "false";
```

```
EOF
```

```
# Abilita gli aggiornamenti automatici
```

```
cat > /etc/apt/apt.conf.d/20auto-upgrades << 'EOF'
```

```
APT::Periodic::Update-Package-Lists "1";
```

```
APT::Periodic::Unattended-Upgrade "1";
```

```
EOF
```

```
sudo systemctl enable unattended-upgrades
```

12.3 Disabilitare login password SSH (solo chiavi)

```
sudo nano /etc/ssh/sshd_config
```

Modifica/aggiungi queste righe:

```

PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
PermitRootLogin no
MaxAuthTries 3

```

```
sudo systemctl restart sshd
```

ATTENZIONE: Prima di fare questo, verifica che il login con chiave SSH funzioni in un terminale separato. Se ti chiudi fuori, devi usare la KVM console di Hetzner.

12.4 Non esporre porte database al pubblico

Le porte 5432 (PostgreSQL), 6379 (Redis), 6333 (Qdrant), 7474/7687 (Neo4j) sono esposte nel docker-compose.yml con ports: - "X:X". Questo le lega a 0.0.0.0 (tutte le interfacce), ma UFW le blocca.

Per sicurezza extra, modifica il compose per bindare solo su localhost:

```

# INVECE DI:
ports:
  - "5432:5432"

# USA:
ports:
  - "127.0.0.1:5432:5432"

```

Questo rende le porte accessibili solo da processi locali (utile per manutenzione), ma non dall'esterno.

12.5 Audit delle porte aperte

```

# Verifica periodicamente le porte in ascolto
sudo ss -tlnp

# Output atteso in produzione:
# Porta 22 — SSH (solo da IP noto idealmente)
# Porta 80 — HTTP → rediretto a HTTPS da Traefik
# Porta 443 — HTTPS (Traefik)
# Porta 9000 — Webhook (opzionale — chiudere se usi Traefik per il webhook)

# Tutte le altre porte devono essere su 127.0.0.1 o non aperte

```

12.6 Proteggere il file .env

```

# .env deve essere leggibile solo dall'utente deploy
chmod 600 /opt/aia-platform/.env
ls -la /opt/aia-platform/.env
# Deve mostrare: -rw----- 1 deploy deploy

```

Il file .env non deve mai essere committed in git. Verifica che .gitignore contenga .env:

```
grep "\.env" /opt/aia-platform/.gitignore
```

12.7 Checklist sicurezza finale

Esegui questo controllo periodicamente (mensile):

```
cat > /opt/aia-platform/security-check.sh << 'EOF'
#!/bin/bash
echo "=== Security Check AIA Platform ==="

echo ""
echo "1. Utenti con shell di login:"
grep -vE '/nologin|false' /etc/passwd | cut -d: -f1

echo ""
echo "2. Fail2ban status:"
fail2ban-client status 2>/dev/null || echo "WARN: fail2ban non attivo"

echo ""
echo "3. UFW status:"
ufw status | head -20

echo ""
echo "4. Aggiornamenti disponibili:"
apt list --upgradable 2>/dev/null | grep -c "upgradable" || true

echo ""
echo "5. Processi in ascolto su 0.0.0.0:"
ss -tlpn | grep '0.0.0.0' | grep -v "127.0.0.1"

echo ""
echo "6. Immagini Docker con vulnerabilità note (richiede docker scout):"
docker compose -f /opt/aia-platform/docker-compose.yml images --format json \
| python3 -c "import sys,json; [print(json.loads(l)['Repository']) for l in sys.stdin if l.strip()]" 2>/dev/null

echo ""
echo "=== Fine Security Check ==="
EOF

chmod +x /opt/aia-platform/security-check.sh
/opt/aia-platform/security-check.sh
```

Appendice A — Comandi di riferimento rapido

```

# === GESTIONE CONTAINER ===
# Avvia tutto
cd /opt/aia-platform && docker compose up -d

# Ferma tutto (SENZA perdere dati)
cd /opt/aia-platform && docker compose stop

# Riavvia un singolo container
docker compose restart litellm

# Stato di tutti i container
docker compose ps

# Log live
docker compose logs -f

# === DEPLOY ===
# Deploy manuale
/opt/aia-platform/deploy.sh

# Log deploy
tail -f /var/log/aia-deploy.log

# === DATABASE ===
# Connessione PostgreSQL interattiva
docker exec -it aia-postgres psql -U aia -d aia_platform

# Backup manuale immediato
/opt/backups/backup-postgres.sh

# === SPAZIO DISCO ===
# Spazio complessivo
df -h

# Dimensione volumi Docker
docker system df

# Pulizia immagini inutilizzate
docker image prune -f

# === MONITORAGGIO ===
# RAM e CPU real-time
docker stats

# Health check completo
/opt/aia-platform/health-check.sh

# === AGGIORNAMENTI ===
# Aggiornamento OS
sudo apt update && sudo apt upgrade -y

# Pull nuove immagini Docker
cd /opt/aia-platform && docker compose pull && docker compose up -d

```

Appendice B — Struttura directory finale sul server

```

/opt/
├── aia-platform/      ← Repository clonato
│   ├── docker-compose.yml ← Infrastruttura (DB, cache, AI, proxy)
│   ├── docker-compose.apps.yml ← Applicazioni (gateway, dashboard, client, downloads)
│   ├── .env          ← SEGRETO — chmod 600
│   ├── deploy.sh     ← Script deploy
│   ├── health-check.sh
│   ├── alert-check.sh
│   ├── security-check.sh
│   ├── public/
│   │   ├── downloads/ ← Binari Desktop Agent (108ai.exe, checksums.txt)
│   │   └── infrastructure/
│   │       ├── traefik/
│   │       ├── litellm/
│   │       ├── postgres/
│   │       ├── redis/
│   │       └── qdrant/
│   ├── backups/
│   │   ├── postgres/ ← Dump giornalieri .sql.gz
│   │   └── volumes/   ← Backup volumi settimanali
│   ├── webhooks/
│   │   └── hooks.json ← Configurazione webhook
│
/var/log/
├── aia-deploy.log    ← Log deploy automatici
└── aia-backup.log    ← Log backup

/home/deploy/
└── webhook-secret.txt ← Segreto webhook (chmod 600)

```

Documento aggiornato il 16 giugno 2026 — 108 Vision

Versione infrastruttura: Traefik v3.1, PostgreSQL 16 + pgvector, Redis 7, Qdrant latest, Neo4j 5 Community, LiteLLM main-latest

Versione applicazioni: Gateway (Node.js 20 + Hono), Dashboard (React 19 + Vite 6 + nginx), Client (React + nginx), Desktop Agent (Bun compile)